

LLM Interview Preparation Complete Guide

For Professionals with ~2 Years of AI/ML Experience

12 Categories	82+ Questions	Interview Ready
--------------------------	--------------------------	----------------------------

Fundamentals | Transformer Architecture | Training & Optimisation | Fine-Tuning & PEFT | RAG & Knowledge |
Prompt Engineering | Agents & Tool Use | Evaluation & Benchmarks | Inference & Deployment | Multimodal
LLMs | Ethics, Safety & Alignment | System Design

From Fundamentals to System Design — One Document to Rule Them All

Table of Contents

■ 1. Fundamentals	10 Q&A;
■ 2. Transformer Architecture	9 Q&A;
■ 3. Training & Optimisation	8 Q&A;
■ 4. Fine-Tuning & PEFT	7 Q&A;
■ 5. RAG & Knowledge	7 Q&A;
■ 6. Prompt Engineering	6 Q&A;
■ 7. Agents & Tool Use	6 Q&A;
■ 8. Evaluation & Benchmarks	6 Q&A;
■ 9. Inference & Deployment	7 Q&A;
■ 10. Multimodal LLMs	4 Q&A;
■ 11. Ethics, Safety & Alignment	6 Q&A;
■ 12. System Design	6 Q&A;

Fundamentals

Q1 What is a Large Language Model (LLM) and how does it differ from traditional NLP models?

ANSWER

An LLM is a neural network trained on massive text corpora (billions-trillions of tokens) to model the probability distribution of language. Unlike traditional NLP models (TF-IDF + logistic regression, early RNNs), LLMs:

- Scale to billions of parameters and exhibit emergent capabilities not seen at smaller scales.
- Are pretrained on diverse data and adapted (fine-tuned/prompted) for downstream tasks without task-specific architectures.
- Use Transformer self-attention to capture long-range dependencies efficiently.
- Demonstrate in-context learning: solving tasks from examples in the prompt alone, zero gradient updates.

Q2 What is the difference between encoder-only, decoder-only, and encoder-decoder models? Give examples.

ANSWER

Encoder-only (BERT, RoBERTa): Bidirectional attention over full input. Produces contextual embeddings. Best for classification, NER, semantic search.

Decoder-only (GPT-4, LLaMA, Mistral, Claude): Causal (left-to-right) attention. Auto-regressive text generation. Dominates chat, code, and reasoning tasks.

Encoder-Decoder (T5, BART, mT5): Encoder reads input bidirectionally; decoder generates output causally, cross-attending to encoder states. Best for translation, summarisation, Q&A; with long contexts.

Q3 What is tokenisation and why does it matter for LLMs?

ANSWER

Tokenisation maps raw text to integer IDs from a fixed vocabulary. Common algorithms:

- BPE (Byte-Pair Encoding): merges frequent character pairs iteratively (GPT-2/3/4, LLaMA).
- WordPiece: uses likelihood gain (BERT).
- SentencePiece: language-agnostic byte-stream approach (T5, mT5).

Why it matters: Token count drives inference cost (attention is $O(n^2)$). Rare words get split into many tokens, increasing length. Languages with non-Latin scripts tokenise less efficiently, inflating costs. Tokenisation artefacts can cause unexpected model behaviours (arithmetic, spelling).

Q4

What is the context window? What are the challenges with very long contexts?

ANSWER

The context window is the maximum number of tokens an LLM can attend to at once (4K for GPT-3, 128K+ for GPT-4o, 1M for Gemini 1.5). Challenges:

- Attention cost scales quadratically: $O(n^2)$ memory and compute.
- 'Lost-in-the-middle' phenomenon: models retrieve information best from beginning and end; middle content is often ignored.
- KV-cache memory grows linearly with context length and batch size.

Solutions: FlashAttention (IO-aware), sliding window attention (Mistral), ring attention, positional extrapolation tricks (YaRN, RoPE scaling), and RAG as an alternative.

Q5

What is the difference between autoregressive and masked language modelling?

ANSWER

Autoregressive LM (GPT-style): Predicts next token given all previous tokens. Training objective: maximise $P(x_t | x_1, \dots, x_{t-1})$. Unidirectional; excellent for generation.

Masked LM (BERT-style): Randomly masks ~15% of tokens and predicts them from bidirectional context. Training objective: maximise $P(x_{\text{masked}} | x_{\text{visible}})$. Better representations for understanding tasks (NLU) but not directly generative. Variants: whole-word masking, span masking (SpanBERT), permutation LM (XLNet).

Q6

What are emergent abilities in LLMs? Give concrete examples.

ANSWER

Emergent abilities are capabilities that appear sharply and unpredictably as model scale increases, absent at smaller scales and not directly trained for. Examples:

- Chain-of-thought reasoning: appears around 100B parameters without explicit CoT training.
- Multi-step arithmetic: models <10B fail consistently; larger models solve reliably.
- Few-shot learning: small models barely improve with examples; large models adapt dramatically.
- Code generation, multilingual translation, instruction following.

Controversy: Wei et al. (2022) documented these; later work (Schaeffer et al., 2023) argues some emergence is a measurement artefact from discontinuous metrics rather than true phase transitions.

Q7 Explain the scaling laws for LLMs. What is the Chinchilla optimal compute frontier?

ANSWER

Scaling laws (Kaplan et al., 2020): LM loss scales as a power law with parameters N , dataset tokens D , and compute C : $L(N) \sim N^{-0.076}$, $L(D) \sim D^{-0.095}$. Original OpenAI work suggested scaling parameters more than data.

Chinchilla (Hoffmann et al., 2022): For compute-optimal training, scale N and D equally. Given compute C , optimal $N \sim C^{0.5}$ and $D \sim C^{0.5}$. Chinchilla (70B, 1.4T tokens) outperformed Gopher (280B, 300B tokens) showing prior models were undertrained.

Implication: LLaMA-series trains smaller models on more tokens for deployment efficiency.

Q8 What is perplexity and what are its limitations as an LLM metric?

ANSWER

Perplexity = $\exp(-1/N * \sum \log P(x_i | \text{context}))$. It measures how surprised the model is by the test corpus; lower is better. Interpretable as: 'how many equally likely choices per token'.

Limitations:

- Sensitive to tokenisation: models with different vocabularies are not directly comparable.
- Doesn't capture downstream task performance (a model with lower perplexity can perform worse on MMLU).
- Doesn't measure factual accuracy, reasoning, or instruction following.
- Distribution mismatch: test-set perplexity may not reflect real-world use cases.
- Cannot evaluate open-ended generation quality.

Q9 What is the difference between zero-shot, few-shot, and in-context learning?

ANSWER

Zero-shot: Model is given a task description with no examples. Relies purely on pre-training knowledge.

Few-shot: N examples (demonstrations) are provided in the prompt. Model adapts its output format and style to match examples without any weight updates. Typical N : 1-8.

In-Context Learning (ICL): The general phenomenon of learning from demonstrations in the prompt. How it works is debated: some argue ICL retrieves/composes learned algorithms; others that it performs implicit Bayesian inference. Key finding: label correctness matters less than format consistency in demonstrations (Min et al., 2022).

Q10

What is temperature in LLM sampling and how does it interact with top-k and top-p?

ANSWER

Temperature T scales the logits before softmax: $p_i = \text{softmax}(z_i / T)$. $T < 1$ sharpens the distribution (more deterministic), $T > 1$ flattens it (more random), $T=0$ gives greedy decoding.

Top-k: Sample from the k tokens with highest probability. Clips the long tail but k is context-insensitive.

Top-p (nucleus sampling): Sample from the smallest set of tokens whose cumulative probability $\geq p$. Adaptive: when distribution is peaked, samples from few tokens; when flat, considers many.

Typical production settings: $T=0.7-1.0$, $\text{top}_p=0.9$, $\text{top}_k=50$. For factual tasks: $T=0$, greedy.

Transformer Architecture

Q11 Explain the multi-head self-attention mechanism mathematically.

ANSWER

For each head h , compute $Q_h = XW_Q^h$, $K_h = XW_K^h$, $V_h = XW_V^h$.

$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) * V$

The scaling factor $\sqrt{d_k}$ prevents vanishing gradients from large dot products.

Multi-head output: $\text{Concat}(\text{head}_1, \dots, \text{head}_H) * W_O$

Why multiple heads: Each head can attend to different positional/semantic relationships. Empirically, heads specialise: some attend locally, some to syntax, some to coreference. H heads with $d_k = d_{\text{model}}/H$ has the same FLOP count as one large head.

Q12 What is Rotary Position Embeddings (RoPE) and why are they preferred?

ANSWER

RoPE (Su et al., 2021) encodes position by rotating query and key vectors in 2D planes. The inner product $Q_m * K_n$ depends only on relative position $(m-n)$, enabling relative position modelling. Advantages over learned absolute positions:

- Relative position modelling naturally.
- Length generalisation beyond training length (with scaling tricks like YaRN).
- No additional parameters.
- Compatible with FlashAttention.

Used in LLaMA, Mistral, Falcon, Gemma, Qwen. ALiBi (linear bias) is an alternative for strong extrapolation at the cost of some perplexity.

Q13 What is Grouped Query Attention (GQA)? How does it compare to MHA and MQA?

ANSWER

MHA: H separate Q, K, V heads. KV-cache size = $2 * H * d_{\text{head}}$ per token. High quality.

MQA: All query heads share a single K, V pair. Smallest KV-cache but quality degrades.

GQA (Ainslie et al., 2023): G groups, each with its own K, V shared among H/G query heads. Interpolates MHA quality and MQA efficiency. KV-cache reduced by factor H/G .

LLaMA 2/3, Mistral, Gemma, Phi-3 all use GQA. At 8 query heads and 1 KV head (MQA-equivalent), you get 8x KV-cache reduction enabling much larger batch sizes and longer contexts.

Q14 What is FlashAttention and why is it important for training and inference?

ANSWER

FlashAttention (Dao et al., 2022) is an IO-aware exact attention algorithm that avoids materialising the full $N \times N$ attention matrix in HBM (GPU DRAM). It tiles computation in SRAM, fusing softmax and GEMMs. Result: $O(n)$ memory (vs $O(n^2)$), 2-4x wall-clock speedup.

FlashAttention-2 adds better parallelism over the sequence dimension and fewer non-GEMM ops. FlashAttention-3 targets Hopper (H100) with async pipelines and FP8.

Critical for: training long-context models (32K+), efficient batch inference, and is now standard in PyTorch 2.0+ via `scaled_dot_product_attention()`.

Q15 What are Mixture-of-Experts (MoE) models? Explain the architecture and trade-offs.

ANSWER

MoE replaces the dense FFN with N expert FFNs and a router that selects top- k experts per token using a gating function: $\text{gate}(x) = \text{softmax}(x * W_g)$. Only k/N of FFN params are active per token.

Examples: Mixtral 8x7B (8 experts, 2 active, 46B total / ~12B active), GPT-4 (rumored MoE), Switch Transformer ($k=1$ for extreme efficiency).

Advantages: More total capacity at same inference FLOPs. Better knowledge compartmentalisation.

Trade-offs: Full parameter set must fit in memory; load balancing (auxiliary loss); communication overhead in distributed settings; training instability; routing collapse risk.

Q16 What is KV-cache and how does it work during autoregressive generation?

ANSWER

During autoregressive generation, each new token attends to all previous tokens. Without caching, K and V for all previous tokens would be recomputed on every step — $O(n^2)$ total compute.

KV-cache stores the Key and Value tensors for all previously generated tokens. On each new step, only the new token's Q , K , V are computed, and K/V are appended to the cache. Attention then uses $[K_{\text{cached}}, K_{\text{new}}]$ and $[V_{\text{cached}}, V_{\text{new}}]$. This reduces generation compute from $O(n^2)$ to $O(n)$.

Memory cost: $2 * L * H * d_{\text{head}} * \text{seq_len} * \text{batch} * \text{dtype_bytes}$. At LLaMA-2-70B with 4096 seq and batch=32, KV-cache is ~160 GB — often the limiting factor for batch size.

Q17 What is the Feed-Forward Network (FFN) in a Transformer? What is SwiGLU?**ANSWER**

Standard FFN: $\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2$, hidden dim = $4 * d_{\text{model}}$.

SwiGLU (Shazeer, 2020): $\text{FFN}(x) = (xW_1 * \text{SiLU}(xW_{\text{gate}})) * W_2$

where $\text{SiLU}(x) = x * \text{sigmoid}(x)$. This is a Gated Linear Unit variant.

Why SwiGLU dominates (LLaMA, PaLM, Mistral):

- Smooth non-linearity (no dead neurons like ReLU)
- Gating enables dynamic feature selection
- Consistent +0.5-1.0 perplexity improvement at matched parameter counts

Note: SwiGLU uses 3 weight matrices vs 2 for standard FFN; hidden dim is typically reduced to $2/3 * 4d$ to keep FLOP count matched.

Q18 What is Layer Normalisation? Compare Pre-LN vs Post-LN Transformer.**ANSWER**

LayerNorm normalises across the feature dimension: $\text{LN}(x) = (x - \text{mean}(x)) / \text{std}(x) * \gamma + \beta$. Unlike BatchNorm, it operates per-sample per-layer, suitable for variable-length sequences.

Post-LN (original Transformer): Residual then LN. Training is unstable without careful warmup; gradients vanish in deep networks.

Pre-LN (modern LLMs): LN applied before sublayer: $x + \text{Sublayer}(\text{LN}(x))$. Much more stable training; allows training without warmup. Used in GPT-2, LLaMA, and most modern models.

RMSNorm (LLaMA): Simplifies to just the scaling part: $\text{RMS}(x) = \sqrt{\text{mean}(x^2)}$, removes mean centering. ~10-15% faster; similar performance.

Q19

What is the difference between sparse attention and dense attention? Name efficient attention variants.

ANSWER

Dense attention: every token attends to all other tokens. $O(n^2)$ memory and compute.

Sparse attention patterns:

- Sliding window (Mistral, Longformer): each token attends to w neighbours. $O(n*w)$.
- Strided (Sparse Transformer): combine local and global stride patterns.
- Sink + window (StreamingLLM): always include first few tokens as 'attention sinks'.
- Dilated attention (LongNet): exponentially increasing gaps.

Approximate attention:

- Linformer: project K, V to lower rank. $O(n)$.
- Performer: random feature approximation of softmax attention. $O(n)$.
- BigBird: combines random, local, and global attention.

In practice, FlashAttention with full attention + long context wins over complex sparse patterns.

Training & Optimisation

Q20 Walk me through the typical LLM training pipeline: pre-training, SFT, and RLHF.

ANSWER

1. Pre-training: Next-token prediction (causal LM loss) on trillions of tokens. The model learns language structure, world knowledge, and reasoning. Very compute-intensive.
2. Supervised Fine-Tuning (SFT): Fine-tune on high-quality prompt-response pairs (FLAN, Alpaca, OpenHermes). Teaches instruction following and assistant-like format.
3. RLHF: Human annotators rank responses -> train a reward model -> use PPO (or DPO) to optimise the LLM against reward with KL-divergence penalty vs. SFT model to prevent reward hacking and language collapse.

Modern pipelines often: SFT -> DPO (skipping explicit RM + PPO), or SFT -> RLAI (Constitutional AI with AI feedback instead of human feedback).

Q21 What is Direct Preference Optimisation (DPO)? How does it compare to PPO?

ANSWER

DPO (Rafailov et al., 2023) derives a closed-form loss from the Bradley-Terry preference model, directly optimising from preference pairs (chosen vs. rejected):

$$L_{DPO} = -\log \text{sigmoid}(\beta * \log(\pi_{\theta}(y_w)/\pi_{\text{ref}}(y_w)) - \beta * \log(\pi_{\theta}(y_l)/\pi_{\text{ref}}(y_l)))$$

Advantages over PPO: No separate reward model, simpler training loop, stable, lower compute.

Disadvantages: Sensitive to data quality; can't explore new responses PPO could; may over-optimize the reference distribution; distribution mismatch between offline preference data and online policy.

Variants: IPO, SimPO (no reference model), ORPO (combined SFT + DPO loss), KTO.

Q22**What is gradient checkpointing and mixed-precision training? Why are they used together?****ANSWER**

Gradient Checkpointing: Instead of storing all intermediate activations for backprop, recompute them during the backward pass. Reduces memory from $O(L)$ to $O(\sqrt{L})$ at ~33% extra compute. Essential for training large models on limited GPU memory.

Mixed Precision (AMP): Compute forward/backward in FP16 or BF16, keep master weights in FP32. BF16 preferred (larger dynamic range than FP16, avoids NaN). Loss scaling prevents underflow. ~2x memory saving and 2-3x throughput.

Used together: checkpointing reduces activation memory, AMP reduces weight/gradient memory. Modern addition: 8-bit optimiser states (bitsandbytes) further reduce memory.

Q23**What are the most common optimisers for LLM training? Why is Adam preferred over SGD?****ANSWER**

Adam (Kingma & Ba, 2014): Maintains per-parameter first moment (gradient mean) and second moment (gradient variance). Adaptive learning rate per parameter. Excellent for sparse gradients and ill-conditioned loss surfaces.

AdamW: Adam + decoupled weight decay (L2 regularisation correctly decoupled from adaptive LR). Standard for LLM training.

SGD: Requires careful LR tuning; works for CNNs but struggles with Transformer optimisation landscape (near-flat regions, sharp curvature).

Memory-efficient alternatives:

- Adafactor: stores factorised second moments; saves 50% memory vs Adam.
- 8-bit Adam (bitsandbytes): quantises optimiser states to 8-bit; 4x memory reduction.
- SOAP, Muon: newer optimisers with better sharpness awareness.

Q24 What is learning rate warmup and cosine decay? Why are they critical for LLM training?

ANSWER

LR Warmup: Gradually increase LR from 0 to target over warmup steps (typically 1-2% of training). Critical because at initialisation, gradients are noisy and large LR causes divergence. Pre-LN Transformers are more robust but warmup still helps.

Cosine Decay: After warmup, decay LR following a cosine curve: $LR(t) = LR_min + 0.5 * (LR_max - LR_min) * (1 + \cos(\pi * t / T))$. Provides gradual annealing allowing model to settle into good minima.

WSD (Warmup-Stable-Decay): Warmup -> long stable phase -> rapid cosine decay. Used in MiniCPM; enables efficient continual learning by adding new data before decay phase.

Q25 What is gradient clipping and why is it important for Transformer training?

ANSWER

Gradient clipping caps the global gradient norm: if $\|g\| > \text{threshold}$, $g = g * (\text{threshold} / \|g\|)$. Typical threshold: 1.0.

Why critical for Transformers: Loss surfaces have sudden sharp curvature (gradient spikes). Without clipping, a single bad batch can cause catastrophic weight updates, diverging training. Clipping provides a safety net without requiring LR to be tiny.

Difference from gradient norm monitoring: Logging the pre-clip norm is a key training health signal. Consistently large norms ($\gg 1$) before clipping indicate the model is struggling or data is corrupted. Sudden spikes often precede loss spikes.

Q26 What is the role of data quality vs quantity in LLM pre-training?

ANSWER

Quantity alone is insufficient. The Chinchilla scaling law shows data and compute must scale together. But quality filtering dramatically improves token efficiency:

- C4 (Raffel et al.): Cleaned Common Crawl with heuristic filters (language detection, content dedup, quality scoring).
- FineWeb (HuggingFace): Aggressive quality filtering yielding ~15T high-quality tokens.
- Phi-1 (Microsoft): 1.3B model trained on 'textbook quality' synthetic data outperformed larger models on coding benchmarks.

Key quality signals: perplexity filtering against a reference model, deduplication (MinHash), toxicity filtering, domain mixing ratios (code, math, web, books).

Q27**What is catastrophic forgetting and what causes it during LLM training?****ANSWER**

Catastrophic forgetting: when learning new tasks/domains causes the model to lose previously learned capabilities (e.g., medical fine-tuning degrades coding ability).

Cause: Gradient descent updates weights to minimise loss on new data, moving away from the optimal weights for prior tasks. The Transformer has no explicit memory of prior tasks.

Mitigation:

- PEFT (LoRA): frozen base weights retain prior knowledge.
- Replay / data mixing: include fraction of pre-training data in fine-tuning mix.
- EWC (Elastic Weight Consolidation): penalise changes to important weights.
- Low learning rate and early stopping.
- Evaluate on general benchmarks (MMLU, HellaSwag) alongside task metrics.

Fine-Tuning & PEFT

Q28

What is LoRA and how does it work mathematically? When should you use it?

ANSWER

LoRA (Low-Rank Adaptation, Hu et al., 2021) freezes pretrained weights W and injects a low-rank update: $W' = W + (\alpha/r) * A*B$, where A in $\mathbb{R}^{(d \times r)}$, B in $\mathbb{R}^{(r \times k)}$, $r \ll \min(d,k)$.

Only A and B are trained ($\sim 0.1\text{-}1\%$ of parameters). At inference, $B*A$ is merged into W , adding zero latency.

When to use: Limited GPU memory, need task-specific adapters swappable without reloading base, mitigate catastrophic forgetting, fast iteration.

Key hyperparameters: rank r (typically 8-64), α (scaling, often $= r$ or $2r$), target modules (usually Q , V , sometimes all attention + FFN).

Q29

Explain QLoRA. What makes it different from LoRA?

ANSWER

QLoRA (Dettmers et al., 2023) enables fine-tuning 65B+ models on a single 48GB GPU:

- 4-bit NormalFloat (NF4): information-theoretically optimal dtype for normally distributed weights. Stores in 4 bits, dequantises block-by-block during compute.
- Double Quantisation: quantise the quantisation constants themselves, saving ~ 0.37 bits/param.
- Paged Optimiser: uses NVIDIA unified memory to page optimiser states to CPU RAM during spikes.

LoRA adapters are trained in BF16 for precise gradients. Base model stays quantised (frozen). QLoRA matches full 16-bit fine-tuning quality while using $\sim 4\times$ less VRAM. Practical: fine-tune LLaMA-2-13B on a single 24GB RTX 3090.

Q30**What are the PEFT methods besides LoRA? Compare Prefix Tuning, Prompt Tuning, and Adapters.****ANSWER**

Prefix Tuning: Prepend trainable 'virtual tokens' to K and V of each attention layer. No change to weights. Good for generation; can be unstable.

Prompt Tuning: Only tune a few tokens prepended to input embedding. Fewest parameters. Performs poorly at <10B params; scales well at very large models.

Adapters: Insert small MLP bottleneck modules after attention/FFN: $x + f(\text{LN}(x))$. Effective but adds sequential latency at inference (cannot be merged like LoRA).

IA3: Rescales K, V, and FFN activations with learned vectors. ~0.01% parameters. Surprisingly effective for classification tasks.

LoRA/QLoRA dominates in practice due to its balance of quality, memory, and zero inference latency.

Q31**What is instruction tuning? How does it differ from task-specific fine-tuning?****ANSWER**

Instruction tuning (Wei et al., 2021 FLAN; Mishra et al., 2021) fine-tunes a model on a diverse collection of NLP tasks expressed as natural language instructions, enabling zero-shot generalisation to unseen tasks.

Key difference from task-specific fine-tuning:

- Task-specific: one model per task, learns format but doesn't generalise.
- Instruction tuning: one model for all tasks, generalises to new task descriptions.

Scale matters: instruction tuning benefits more from task diversity than dataset size. FLAN-T5 (1.8K task templates) outperforms T5 fine-tuned on individual tasks. Datasets: FLAN, Super-NaturalInstructions, OpenHermes, ShareGPT, UltraChat.

Q32 What is ORPO and how does it simplify the alignment pipeline?

ANSWER

ORPO (Odds Ratio Preference Optimisation, Hong et al., 2024) combines SFT and DPO into a single training stage. The loss has two components:

- SFT loss: standard negative log-likelihood on chosen responses.
- Odds ratio penalty: discourages rejected responses using the log odds ratio between chosen and rejected, without a separate reference model.

$$L_{\text{ORPO}} = L_{\text{SFT}} - \lambda * \log(\text{sigmoid}(\log(\text{odds}(\text{chosen})) - \log(\text{odds}(\text{rejected}))))$$

Advantages: No separate SFT stage, no reference model (saves GPU memory), single training run. Similar or better alignment quality than two-stage SFT + DPO. Used for Phi-3, Qwen.

Q33 How do you select which layers and modules to apply LoRA to?

ANSWER

Standard practice: apply to query (q_proj) and value (v_proj) projection matrices. This follows the original LoRA paper and covers most adaptation with minimum parameters.

Expanded targets: add key (k_proj), output (o_proj), and FFN layers (gate_proj, up_proj, down_proj) for more expressive adaptation. Needed for significant domain shifts.

Rank selection: r=8-16 for lightweight tasks; r=32-64 for complex tasks or domain shifts. Higher rank = more capacity but more parameters and overfitting risk on small datasets.

Tools: PEFT library (HuggingFace) handles target module selection automatically with task-type presets. LoRA-rank search: use rsLoRA (rank-stabilised) which normalises by $\sqrt{r}$ instead of r , enabling stable training at high ranks.

Q34 What dataset size is sufficient for instruction fine-tuning with LoRA?

ANSWER

Less than you'd think. Key findings from research:

- Alpaca (Stanford): 52K GPT-3.5 generated samples transformed GPT-J into an instruction follower. Quality > quantity.
- LIMA (Zhou et al., 2023): Only 1000 carefully curated examples matched ChatGPT-level instruction following. 'Superficial alignment hypothesis.'
- General rule: 1K-10K high-quality examples for style/format adaptation; 10K-100K for domain knowledge injection.

Quality signals: diversity of instructions, clear input-output mapping, no contradictions, appropriate length, factual accuracy. Decontaminate eval benchmarks from training data. Data curation > data volume for fine-tuning.

RAG & Knowledge

Q35 What is Retrieval-Augmented Generation (RAG)? When to use RAG vs fine-tuning?

ANSWER

RAG grounds LLM responses in an external knowledge base:

1. Query -> embedding model -> vector search -> top-k chunks.
2. Chunks + query -> LLM -> grounded answer.

Use RAG when: knowledge changes frequently, data is proprietary, you need citations, training budget is limited, or knowledge is too large for context.

Use fine-tuning when: changing model behaviour/style/format, teaching new reasoning patterns, domain vocabulary not in base model.

They're complementary: fine-tune for style/format, RAG for fresh factual grounding. Modern systems use both (e.g., fine-tune for citation style, RAG for content).

Q36 Describe the full RAG pipeline and key failure modes.

ANSWER

Pipeline: Document ingestion -> chunking (fixed/semantic/agentive) -> embedding (sentence-transformers, OpenAI ada-002, BGE) -> vector store (FAISS, Pinecone, Weaviate, pgvector) -> retrieval (ANN + optional reranking) -> context assembly -> LLM generation.

Failure modes:

- Retrieval failure: relevant chunk not in top-k (fix: better chunking, hybrid search).
- Chunking errors: splitting mid-sentence or mid-table loses context.
- Embedding mismatch: query/document embeddings in different semantic spaces.
- Lost-in-the-middle: LLM ignores middle chunks.
- Hallucination despite retrieval: LLM ignores retrieved context.
- Stale index: documents updated but embeddings not refreshed.
- Cascading errors: retrieval error + LLM confabulation.

Q37 What is hybrid search? Explain BM25 + dense retrieval with RRF fusion.

ANSWER

Hybrid search combines sparse (keyword) and dense (semantic) retrieval:

- BM25 (sparse): TF-IDF-like; excellent for exact keyword/entity matching. $\text{Score}(q,d) = \sum \text{IDF}(t) * f(t,d) * (k_1+1) / (f(t,d) + k_1 * (1-b+b * |d|/\text{avgdl}))$.

- Dense retrieval (FAISS + sentence-transformers): semantic similarity even when keywords differ.

Reciprocal Rank Fusion (RRF): $\text{RRF}(d) = \sum_i 1/(k + \text{rank}_i(d))$, $k=60$.

Robust: no score normalisation needed; handles different score scales from sparse/dense.

After fusion, a cross-encoder reranker (ms-marco-MiniLM, Cohere Rerank) further refines top-k results at higher computational cost but significant quality gain.

Q38 What are chunking strategies in RAG? Compare fixed-size, semantic, and recursive chunking.

ANSWER

Fixed-size chunking: Split every N tokens with M token overlap. Simple but may cut mid-sentence. Good baseline for homogeneous documents.

Recursive character text splitter (LangChain default): Try to split on paragraphs, then sentences, then words, then characters. Respects natural boundaries.

Semantic chunking: Embed sentences, detect embedding similarity drops to find natural topic boundaries. Chunks are semantically coherent but variable size.

Agentic/hierarchical chunking: Create parent chunks (full section) and child chunks (sentences). Retrieve child chunks, return parent for context (parent-child retrieval).

Specialised: Table-aware (extract tables separately), code-aware (function-level), PDF layout-aware (preserve columns, headers). Chunk size 512-1024 tokens typical.

Q39 What is ColBERT and how does it improve retrieval over bi-encoder models?

ANSWER

Bi-encoder (standard dense retrieval): Embed query Q and document D to single vectors; similarity = $\text{dot}(Q, D)$. Very fast (precomputed doc embeddings, ANN search) but compresses all information into one vector (information bottleneck).

ColBERT (Khattab & Zaharia, 2020): Late interaction model. Embed Q to $|Q|$ token vectors, D to $|D|$ token vectors. Score = $\sum_{q \in Q} \max_{d \in D} q^T \cdot d$ (MaxSim).

Advantages: Much richer matching (each query token finds its best document token match). Competitive with cross-encoders at bi-encoder speed (with PLAID engine).

Trade-off: Larger index (one vector per token vs one per doc), more complex ANN structure. ColBERTv2 reduces index size 6-10x via compression.

Q40 What is GraphRAG and when does it outperform standard vector RAG?

ANSWER

GraphRAG (Microsoft, 2024) builds a knowledge graph from the document corpus (entity extraction, relationship extraction, community detection) and uses graph-structured retrieval alongside or instead of vector search.

Two modes:

- Local search: traditional entity-centric Q&A, augmented with graph context.
- Global search: community summaries enable dataset-wide reasoning ('what are the main themes across all documents?').

Outperforms standard RAG for: holistic document understanding, multi-hop reasoning, relationship queries ('how does X connect to Y?'), cross-document synthesis.

Trade-offs: Expensive graph construction (many LLM calls), higher latency, requires entity/relation schema design. Overkill for factoid Q&A.

Q41

What is FLARE and Self-RAG? How do they improve over naive RAG?

ANSWER

Naive RAG: retrieve once at query time -> generate. Works for simple factoid Q&A.;

FLARE (Active Retrieval Augmented Generation): Generates text token-by-token; when generation confidence drops below threshold, pauses and retrieves additional context using the partial generation as query. Multi-hop and iterative retrieval.

Self-RAG (Asai et al., 2023): Trains the LLM to output special reflection tokens: [Retrieve]: should I retrieve? [ISREL]: is retrieved passage relevant? [ISSUP]: does passage support generation? [ISUSE]: is response useful? Model learns to self-evaluate and selectively retrieve, improving both quality and efficiency.

Both improve factuality on knowledge-intensive tasks at the cost of latency.

Prompt Engineering

Q42 What is Chain-of-Thought prompting? What are its variants?

ANSWER

CoT prompting encourages the model to articulate intermediate reasoning steps before producing an answer ('Let's think step by step'). This leverages the model's ability to self-condition on its generated tokens.

Variants:

- Zero-shot CoT: just append 'Let's think step by step'.
- Few-shot CoT: provide complete (question, reasoning chain, answer) examples.
- Self-Consistency CoT: sample N CoT reasoning paths, take majority vote answer.
- Tree of Thoughts (ToT): branch reasoning into a tree, evaluate intermediate states, use BFS/DFS to find best path.
- Program of Thoughts (PoT): generate Python code instead of natural language reasoning; execute for precise arithmetic.

Helps: arithmetic, logical deduction, multi-step reasoning. Fails: factual recall tasks, hallucination-prone domains.

Q43 What is prompt injection and how do you defend against it?

ANSWER

Prompt injection: attacker embeds instructions in user input or retrieved content to override system instructions.

Types:

- Direct: user directly instructs 'ignore previous instructions and...'
- Indirect: malicious content in retrieved documents/tool outputs hijacks the agent.

Defence strategies:

- Privilege separation: use separate API calls for trusted/untrusted content.
- Input/output filtering: classifiers or regex patterns for injection attempts.
- Structured outputs: JSON mode / grammar-constrained decoding limits output space.
- Sandboxing: run LLM-triggered actions in restricted environments.
- Human-in-the-loop for high-stakes actions.
- Canary tokens: detect if secret strings appear in outputs.
- Prompt hardening: explicitly instruct model to ignore conflicting instructions in content.

Q44

What are system prompts and how do they interact with user prompts and assistant context?

ANSWER

System prompt: Sets model persona, capabilities, constraints, and response format. Sent before user messages. Processed by the model as high-priority context.

Interaction hierarchy: System > User > Assistant (in most fine-tuned models). Well-aligned models treat system instructions as persistent constraints.

Best practices:

- Be explicit and specific: 'You are a customer support agent for Acme Corp. Always respond in formal English. Never reveal internal pricing.' vs vague persona descriptions.
- Include output format specification (JSON schema, markdown structure).
- Specify handling of edge cases ('If user asks about competitors, politely decline').
- Keep it concise: very long system prompts dilute attention to later instructions.

Note: System prompt contents can often be extracted via prompt injection attacks.

Q45

What is structured output generation (JSON mode) and how is it implemented?

ANSWER

Structured output forces LLMs to generate valid JSON (or other structured formats), eliminating parsing failures in production pipelines.

Implementation approaches:

- Prompt-based: 'Respond only in valid JSON with schema {...}'. Unreliable.
- Logit masking (grammar-constrained decoding): At each token step, mask logits for tokens that would produce invalid JSON given current state (via a grammar automaton). Used in Outlines, llama.cpp grammar, SGLang. 100% valid JSON guaranteed.
- Fine-tuned JSON mode: OpenAI, Anthropic, Gemini native JSON mode uses training + constrained decoding.

Libraries: Outlines (grammar-constrained), Instructor (retry loop + Pydantic), LangChain output parsers, Marvin, Kor.

Q46

What is ReAct prompting and how does it enable reasoning + action?**ANSWER**

ReAct (Yao et al., 2022) interleaves Reasoning traces and Actions in the LLM output. Format: Thought -> Action -> Observation -> Thought -> Action...

Example trace:

Thought: I need to find the population of Tokyo.

Action: search('Tokyo population 2024')

Observation: Tokyo has 13.96 million residents.

Thought: Now I can answer the question.

Action: finish('13.96 million')

Advantages: Combines LLM reasoning with tool use; interpretable reasoning chain; errors are diagnosable via the trace. Foundation for modern agent frameworks (LangChain, LlamaIndex agents, AutoGPT). Outperforms standalone CoT on tasks requiring external knowledge.

Q47

How do you optimise prompts systematically? What is DSPy?**ANSWER**

Manual prompt optimisation: A/B testing variants, systematic ablation, eval-driven iteration.

Automatic Prompt Optimisation:

- APE (Automatic Prompt Engineer): Generate prompt variants with LLM, score on eval set, select best.
- OPRO (Google): Use LLM as optimiser; provide past prompts + scores, ask for improvements.

DSPy (Khatab et al., 2023): Framework that compiles declarative LLM programs into optimised prompts/fine-tuning. You write a program with Modules (Predict, ChainOfThought, RAG) and Metrics; DSPy's Teleprompter optimises prompts/demonstrations automatically. Treats prompt engineering as a software engineering problem: declarative, testable, composable. Particularly powerful for multi-step pipelines where manual tuning is intractable.

Agents & Tool Use

Q48

What is an LLM agent? Describe its core components.

ANSWER

An LLM agent uses an LLM as the reasoning engine to autonomously plan and execute multi-step tasks by calling tools, observing results, and iterating.

Core components:

- LLM (brain): reasoning, planning, decision-making.
- Tools/Actions: web search, code execution, database queries, API calls, file I/O.
- Memory: short-term (context window), long-term (vector store/external DB).
- Planning: decompose task into subtasks (ReAct loop, plan-and-execute, hierarchical).
- Observation: parse tool outputs and feed back to LLM.

Frameworks: LangChain Agents, LlamaIndex Agents, AutoGPT, BabyAGI, CrewAI (multi-agent), Semantic Kernel, OpenAI Assistants API.

Q49

What is function calling (tool use) in LLMs? How does it work at the API level?

ANSWER

Function calling allows LLMs to output structured calls to predefined functions instead of plain text. The model decides when and which function to call.

API flow (OpenAI/Anthropic style):

1. Developer provides tool schemas (JSON Schema: name, description, parameters).
2. LLM receives user message + tool schemas.
3. LLM outputs a tool_call message with function name and arguments (JSON).
4. Developer executes the function, gets result.
5. Result is added to conversation as tool_result message.
6. LLM generates final response incorporating tool output.

Implementation: Fine-tuned on (prompt, tool_call, observation, response) trajectories. Constrained decoding ensures valid JSON in tool arguments.

Q50**What is the difference between plan-and-execute and ReAct agent architectures?****ANSWER**

ReAct (Interleaved): Think -> Act -> Observe, one step at a time. The plan is implicit and emerges from the reasoning trace. Simple, works well for short tasks. Prone to getting stuck in loops or losing track of the overall goal.

Plan-and-Execute: Explicit two phases. Planner LLM creates a full plan (subtask list). Executor LLM executes each subtask. Replanning triggered when execution fails.

Advantages of Plan-and-Execute: Better for long-horizon tasks; plan is inspectable/editable; can parallelise independent subtasks; reduces context growth.

Hybrid (LangGraph, AutoGen): Stateful graph where nodes are LLM calls or tools; edges are conditional transitions. Supports cycles, branching, and human-in-the-loop nodes.

Q51**What are multi-agent systems? When do you use them over a single agent?****ANSWER**

Multi-agent: Multiple specialised LLM agents collaborate, each with a specific role, toolset, and expertise. Orchestrator agent delegates subtasks.

Patterns:

- Supervisor: Central agent delegates to specialised subagents (researcher, coder, writer).
- Peer-to-peer: Agents communicate directly (AutoGen conversation pattern).
- Hierarchical: Nested orchestration (meta-agent -> domain agents -> tool agents).

When to use multi-agent:

- Task exceeds single context window.
- Parallelisable subtasks (map-reduce style).
- Specialised expertise needed (code agent + data analysis agent + report agent).
- Quality checks via disagreement/debate between agents.

Challenges: Coordination overhead, error propagation, cost multiplication, debugging difficulty.

Q52**What is Model Context Protocol (MCP) and why is it significant?****ANSWER**

MCP (Anthropic, 2024) is an open standard protocol for connecting LLMs to external tools and data sources. It defines a client-server architecture where:

- MCP Server: exposes resources (files, DB, APIs) and tools via standardised JSON-RPC.
- MCP Client (LLM app): discovers and calls server capabilities at runtime.

Significance: Before MCP, every LLM app needed custom integrations for each tool. MCP creates an ecosystem where tools written once work with any MCP-compatible client (Claude, Claude Code, Cursor, etc.). Like USB-C for AI tool integrations.

Components: Resources (read-only data), Tools (callable functions), Prompts (reusable prompt templates).
Transport: stdio (local) or SSE/HTTP (remote).

Q53**How do you handle tool errors and agent loops in production agent systems?****ANSWER**

Tool errors:

- Retry with exponential backoff for transient failures (rate limits, timeouts).
- Provide error message to LLM and let it decide: retry, use different tool, or ask user.
- Set max_retries (typically 3) to prevent infinite retry loops.

Agent loops:

- Hard limits: max_iterations (20-50) and max_execution_time.
- Loop detection: track (tool, args) pairs; if repeated 3x, force termination.
- Progress check: after N steps, ask LLM 'Are you making progress toward the goal?'

Cost controls:

- Token budget per agent run.
- Escalation to human-in-the-loop when uncertainty is high.
- Logging every LLM call + tool call for post-hoc debugging.
- Async/parallel execution for independent tool calls.

Evaluation & Benchmarks

Q54 How do you evaluate an LLM? Describe key automatic and human evaluation approaches.

ANSWER

Automatic metrics:

- Perplexity: predictive quality on held-out corpus. Good for pre-training comparisons.
- BLEU/ROUGE: n-gram overlap; useful for translation/summarisation, poor for open-ended.
- BERTScore: contextual embedding similarity.
- Benchmarks: MMLU (57-subject knowledge), HumanEval (code), GSM8K (math), HellaSwag (commonsense), ARC (reasoning), TruthfulQA (factuality), LMSYS Arena (ELO).
- LLM-as-judge: GPT-4/Claude grades responses (fast, scalable, but has positional and verbosity biases).

Human evaluation: preference ratings, Likert scales, A/B testing. Gold standard but expensive and slow. LMSYS Chatbot Arena uses crowd-sourced blind preference voting.

Q55 What is hallucination in LLMs? What are the types and mitigation strategies?

ANSWER

Hallucination: the model generates plausible-sounding but factually incorrect content.

Types:

- Intrinsic: contradicts the provided source/context.
- Extrinsic: content cannot be verified against source.
- Factual: incorrect world knowledge claims.
- Reasoning: incorrect logical deductions.

Mitigations:

- RAG: anchor responses to retrieved documents.
- RLHF with factuality reward signals.
- Self-consistency: sample multiple responses, detect disagreements.
- Citation generation: force model to cite source passages.
- Uncertainty quantification: 'I'm not sure' threshold.
- Retrieval-augmented evaluation (RAGAS) to detect unsupported claims.

Q56**What is LLM-as-judge? What are its biases and how do you mitigate them?****ANSWER**

LLM-as-judge uses a powerful model (GPT-4, Claude) to evaluate other LLM responses, scoring on criteria like helpfulness, accuracy, safety, coherence.

Common biases:

- Positional bias: prefers response A when shown as first, regardless of quality.
- Verbosity bias: prefers longer responses even if less accurate.
- Self-enhancement bias: GPT-4 prefers GPT-4-style responses.
- Sycophancy: agrees with the human's implied preference.

Mitigations:

- Swap positions and average scores.
- Use reference answers ('judge with reference').
- Calibrate with human labels.
- Use multiple judge models and ensemble.
- Provide explicit scoring rubrics with examples.
- MT-Bench uses GPT-4 with reference answers and averaged position-swapped scores.

Q57**What is RAGAS and how do you evaluate a RAG pipeline?****ANSWER**

RAGAS (Retrieval-Augmented Generation Assessment) is a framework for evaluating RAG pipelines across four dimensions:

- Faithfulness: Is the answer factually consistent with the retrieved context? (fraction of answer claims supported by context)
- Answer Relevance: Does the answer address the question? (cosine sim of question and generated questions from answer)
- Context Precision: Are retrieved chunks actually relevant to the question? (signal-to-noise ratio in retrieved context)
- Context Recall: Does the retrieved context contain necessary information to answer? (requires ground truth answer)

RAGAS uses LLM-based evaluation for each metric. Combine with retrieval metrics: MRR, NDCG@k, Hit Rate@k using a golden Q&A; evaluation set.

Q58**What is the difference between benchmark contamination and data leakage?****ANSWER**

Benchmark contamination: Test benchmark examples (or very similar content) appear in the model's pre-training corpus, inflating scores. Example: GPT-4 performing well on HumanEval partly because GitHub code (including solutions) was in the training data.

Detection: n-gram overlap analysis between training data and benchmark; memorisation tests (exact recall of test problems); train/test split temporal analysis.

Data leakage (broader): Any information from the test set leaking into training/development. Can be indirect (test labels used to select model hyperparameters = multiple testing bias).

Mitigations: Dynamic/private benchmarks (no public test set); LLM eval on held-out data curated after training cutoff; decontamination pipelines (remove 13-gram matches).

Q59**How do you evaluate LLMs for production reliability? What metrics matter in real systems?****ANSWER**

Production metrics beyond benchmark accuracy:

- Latency: TTFT (Time to First Token) for perceived responsiveness; total response time.
- Throughput: tokens/second; requests/second at target latency SLO.
- Reliability: refusal rate (when should model decline?), error rate, hallucination rate measured on golden evaluation set weekly.
- Instruction following: does the model respect format/length/style constraints?
- Consistency: same query -> same quality answer across runs (temperature=0 for testing).
- Safety: harmful output rate on red-teaming test set.
- Cost: \$/1K tokens; cost per task completion.

Tooling: LangSmith, Weights & Biases, Arize Phoenix, MLflow for LLM observability.

Inference & Deployment

Q60 Explain the key LLM inference optimisation techniques: quantisation, batching, speculative decoding.

ANSWER

Quantisation: Reduce weight precision (FP16 -> INT8 -> INT4). GPTQ, AWQ, GGUF are popular formats. INT4 gives ~4x memory reduction with <2% quality loss on most tasks.

Continuous Batching: Unlike static batching, continuously insert new requests as slots free up. Dramatically improves GPU utilisation (vLLM, TGI).

PagedAttention (vLLM): Manages KV-cache in non-contiguous memory pages like an OS, eliminating KV-cache fragmentation and enabling higher throughput.

Speculative Decoding: A small 'draft' model proposes K tokens; the large model verifies all K in one forward pass. Accepts agreeing tokens, discards at first mismatch. 2-4x latency improvement with no quality loss.

Q61 What is vLLM and how does PagedAttention work?

ANSWER

vLLM is an open-source LLM inference server built around PagedAttention. Traditional inference pre-allocates contiguous KV-cache for `max_seq_len x batch_size`, wasting memory for short sequences and causing fragmentation.

PagedAttention partitions the KV-cache into fixed-size pages (blocks), mapped via a block table — identical to virtual memory in OS kernels. Benefits:

- Near-zero KV-cache waste (<4% fragmentation).
- Copy-on-write for parallel sampling/beam search (share prefix pages).
- 24x higher throughput vs. naïve HuggingFace Transformers at same latency SLO.

Additional vLLM features: prefix caching (reuse KV for shared system prompts), chunked prefill, disaggregated prefill/decode (DistServe), speculative decoding.

Q62**What is model quantisation? Compare GPTQ, AWQ, and GGUF.****ANSWER**

Quantisation reduces model weight precision to decrease memory and increase throughput.

GPTQ (Frantar et al., 2022): Post-training quantisation. Uses second-order (Hessian) information to minimise quantisation error layer by layer. 3-4 bit quantisation of 175B GPT models. Requires calibration dataset. Good GPU inference.

AWQ (Lin et al., 2023): Activation-aware Weight Quantisation. Scales weights based on activation magnitude before quantising — protects salient channels. 4-bit, slightly better than GPTQ at same bit width. Hardware-friendly implementation.

GGUF (llama.cpp): CPU-friendly format for running quantised models on consumer hardware (M1/M2 Mac, CPU). Supports Q2_K to Q8_0 quantisation. Popular for local deployment. Slower than GPU-based GPTQ/AWQ but accessible without expensive hardware.

Q63**What is tensor parallelism and pipeline parallelism for LLM training and inference?****ANSWER**

Tensor Parallelism (TP): Split individual weight matrices across GPUs. For attention: each GPU handles a subset of heads. For FFN: split along hidden dimension. Requires all-reduce communication after each layer. Low latency but high bandwidth needed. Used for single-node multi-GPU (NVLink). Example: Megatron-LM.

Pipeline Parallelism (PP): Split model layers across GPUs/nodes. GPU 1 processes layers 1-8, GPU 2 processes layers 9-16, etc. Lower bandwidth need but introduces pipeline bubbles (idle time between microbatches). Mitigated by 1F1B schedule or interleaved stages.

Sequence Parallelism: Split sequence dimension across GPUs for very long contexts.

3D Parallelism (Megatron-DeepSpeed): TP + PP + Data Parallelism for trillion-parameter models.

Q64**How do you serve LLMs cost-effectively? Explain token streaming, caching, and model routing.****ANSWER**

Token streaming: Return tokens as they're generated via SSE/WebSocket. Dramatically improves perceived latency (users see first token in ~200ms vs waiting 10s for full response). Standard in all production APIs.

Semantic caching: Cache (query_embedding -> response) pairs. On new query, check cosine similarity to cached queries; if >0.95 similarity, return cached response. 10-30% cache hit rates typical. Tools: GPTCache, Redis with vector similarity.

KV-prefix caching: Cache KV tensors for shared system prompts. vLLM supports this natively, reducing prefill cost by 50-80% for high-traffic prompts.

Model routing: Route simple queries (greetings, factoid) to small fast models (Haiku, Mistral-7B); complex queries to large models. Classifier-based or LLM-based routing. Can reduce costs 60-80% with minimal quality loss.

Q65**How would you design an LLM production system for 10,000 QPS?****ANSWER**

1. Load Balancer: Distribute requests across inference replicas. Sticky routing for multi-turn sessions.
2. Inference Fleet: GPU clusters (A100/H100) running vLLM with continuous batching. Autoscale on GPU queue depth metric via Kubernetes HPA.
3. Caching: Semantic cache (Redis + vector similarity) for repeated queries; KV-prefix cache for shared system prompts.
4. Rate Limiting: Token-bucket per tenant; protect against prompt flooding.
5. Model Routing: Small model (Haiku/GPT-4o-mini) for simple queries; large model for complex.
6. Observability: Log TTFT, TBT (time-between-tokens), throughput, GPU utilisation, error rates via LangSmith/Datadog.
7. Safety Layer: Input/output classifiers before/after LLM call.
8. Fallback: Degrade to smaller model under load; queue and retry for batch tasks.

Q66**What is disaggregated prefill-decode and why does it matter?****ANSWER**

In standard LLM serving, prefill (processing the prompt) and decode (generating tokens) run on the same GPU. They have fundamentally different compute profiles:

- Prefill: compute-bound (processes all tokens in parallel, high GPU utilisation).
- Decode: memory-bandwidth-bound (one token at a time, low GPU utilisation).

Disaggregated serving (Splitwise/DistServe): Separate GPU pools for prefill and decode. Prefill pool: fewer, compute-optimised GPUs. Decode pool: more, memory-bandwidth-optimised GPUs.

Benefits: 2-3x throughput improvement; better hardware utilisation; decode pool can be cheaper (older GPUs). Trade-off: KV-cache transfer between pools adds latency and requires high-bandwidth interconnect.

Multimodal LLMs

Q67 How do Vision-Language Models (VLMs) work? Explain the architecture.

ANSWER

VLMs combine a vision encoder with an LLM decoder:

1. Vision Encoder: ViT (Vision Transformer) encodes image patches to embeddings. CLIP ViT-L/14 or ViT-G/14 are common choices (trained with contrastive image-text loss).
2. Projection Layer: Linear layer or MLP maps vision embeddings to the LLM's token embedding space (e.g., LLaVA uses a simple linear projection; InternVL uses a 'dynamic resolution' approach with pixel shuffling).
3. LLM Decoder: Processes interleaved image tokens and text tokens.

Training: Two-stage — (1) pretrain projection layer on image-caption pairs (encoder+LLM frozen); (2) instruction tune full model on multimodal QA data.

Examples: LLaVA-1.6, InternVL2, Qwen-VL, Claude claude-opus-4-6, GPT-4V, Gemini.

Q68 What is CLIP and how is it used in multimodal systems?

ANSWER

CLIP (Contrastive Language-Image Pre-training, OpenAI 2021) trains image and text encoders jointly with contrastive loss: maximise cosine similarity of matched (image, text) pairs and minimise similarity of mismatched pairs (InfoNCE loss).

Training data: 400M (image, alt-text) pairs from the web. Result: shared embedding space where similar concepts cluster regardless of modality.

Applications:

- Zero-shot image classification: embed labels as text, classify by nearest text embedding.
- Image retrieval: text query -> retrieve images by cosine similarity.
- VLM vision encoder: CLIP ViT provides rich visual features for LLaVA, Stable Diffusion, etc.
- Reward model for image generation (CLIP score).

Limitation: weak at fine-grained text reading (OCR), spatial reasoning, counting.

Q69**What are the challenges of processing high-resolution images in VLMs?****ANSWER**

Standard ViT processes images as fixed-size patches (224x224 -> 196 patches at 16x16). High-resolution images cause:

- Token explosion: 1024x1024 -> 4096 patches; attention cost is $O(n^2)$ -> impractical.
- Resolution mismatch: downsampling loses fine-grained detail (text, small objects).

Solutions:

- Dynamic resolution (InternVL, LLaVA-HD): Tile image into smaller crops, encode each separately, concatenate tokens. Thumbnail for global context + tiles for detail.
- Pixel shuffle (InternVL2): 4x4 patch merge reduces token count 4x while preserving info.
- AnyRes (LLaVA-1.6): Select grid configuration (1x2, 2x2, etc.) matching image aspect ratio.
- Linear projection + MLP (Flamingo): Cross-attention from text to vision features, reducing visual tokens via Perceiver Resampler.

Q70**What is chain-of-thought for vision tasks? How do VLMs handle spatial reasoning?****ANSWER**

VLMs can use text-based CoT for visual reasoning: 'First, I observe X in the image. Then I note Y. Therefore...'. This works for:

- Multi-step visual question answering.
- Chart/graph interpretation.
- Diagram understanding.

Spatial reasoning challenges: VLMs struggle with precise spatial relationships (left/right, above/below, counts), bounding box prediction, and pixel-level tasks.

Solutions:

- Set-of-Mark prompting: overlay numbered marks on image regions; reference by number.
- Chain-of-thought with explicit coordinate reasoning.
- Specialised models: Grounding DINO for detection; SAM for segmentation; GPT-4V with coordinate prompting.
- Training on spatial reasoning datasets (SpatialBench, What's Up).

Ethics, Safety & Alignment

Q71 What is Constitutional AI (CAI) and how does Anthropic use it?

ANSWER

Constitutional AI (Bai et al., 2022) trains AI systems to be helpful, harmless, and honest using a set of principles ('constitution') without relying on human labelling of every harmful response.

Process:

1. SL-CAI: Model critiques and revises its own responses according to the constitution.
2. RL-CAI: A preference model trained on AI-generated feedback is used to fine-tune the policy via RL.

Benefits: Scales safety training cheaply; makes safety criteria explicit and auditable; reduces harmful outputs without excessive refusals. Claude models are trained with CAI.

The constitution includes principles like: 'Choose the response that is less harmful'; 'Choose the response that is most honest'; 'Prefer responses that respect human autonomy'.

Q72 What is reward hacking and sycophancy in RLHF?

ANSWER

Reward hacking: The model finds ways to maximise the reward model's score without actually being more helpful. Examples: generating long, confident-sounding responses even if incorrect; using flattery; formatting tricks.

Sycophancy: Model agrees with users' stated or implied preferences even when wrong. If a user expresses a false belief, a sycophantic model validates it. Arises because human raters prefer responses that agree with their views.

Mitigations:

- KL divergence penalty: constrains policy from drifting too far from SFT model.
- Diverse reward models (ensemble).
- Red-teaming for reward hacking patterns.
- Factuality-specific reward signals.
- Constitutional AI explicitly includes anti-sycophancy principles.

Q73**What is jailbreaking? Describe common attack types and defences.****ANSWER**

Jailbreaking: adversarial prompts that bypass an LLM's safety guardrails.

Attack types:

- Role-play attacks: 'Pretend you are DAN (Do Anything Now)...'
- Hypothetical framing: 'In a fictional world where...'
- Encoding attacks: base64/leetspeak/pig Latin obfuscation of harmful content.
- Many-shot jailbreaking: flood context with examples of compliant harmful responses.
- Gradient-based (GCG): automatically find adversarial suffixes that transfer across models.
- Prompt injection via retrieved content.

Defences:

- RLHF/RLAIF safety training; adversarial fine-tuning (include jailbreak attempts in training).
- Input classifiers (Llama Guard, OpenAI Moderation API).
- Output classifiers before delivering response.
- Prompt certification (detect adversarial patterns).
- SmoothLLM: randomise input perturbations, ensemble predictions.

Q74**What is the alignment problem? Explain outer and inner alignment.****ANSWER**

The alignment problem: ensuring AI systems pursue goals that are beneficial to humanity, not just objectives specified during training.

Outer alignment (specification problem): The specified objective doesn't perfectly capture what we actually want. Goodhart's Law: 'When a measure becomes a target, it ceases to be a good measure.' RLHF reward models are imperfect proxies for human values.

Inner alignment (mesa-optimisation problem): Even if the outer objective is correct, the learned model (mesa-optimiser) may pursue a different internal objective that generalises differently. The model may appear aligned during training but misalign at deployment.

Current mitigations: Constitutional AI, debate, amplification, interpretability research, scalable oversight, process-based supervision (reward reasoning not just outcomes).

Q75 What is red-teaming for LLMs and how is it done?

ANSWER

Red-teaming: systematic adversarial testing to find safety failures, harmful outputs, and jailbreaks before deployment.

Manual red-teaming: Human experts probe the model across harm categories (violence, CSAM, CBRN weapons, self-harm, bias, privacy). Used by Anthropic, OpenAI for model releases.

Automated red-teaming:

- LLM-based: Use a red-team LLM to generate adversarial prompts, target LLM generates responses, judge LLM scores harm. Iterate.
- Gradient-based (GCG, AutoDAN): Optimise adversarial suffix via gradient information.
- Multi-turn: Test safety in extended conversations, not just single turns.

Harm taxonomy: Meta's Llama Guard uses 14 categories from the MLCommons taxonomy (violent crimes, CSAM, privacy, misinformation, etc.) as classification schema.

Q76 What are the key LLM safety risks a practitioner should know and monitor?

ANSWER

Model-level risks:

- Hallucination / factual errors: false information presented confidently.
- Bias & fairness: stereotyping, demographic disparities in quality.
- Jailbreaking: safety bypass via adversarial prompts.
- Over-refusal: excessive caution makes model unhelpful for legitimate use cases.
- Copyright / PII leakage: memorised training data reproduced at inference.

System-level risks:

- Prompt injection via retrieved/tool content.
- Agent autonomy: agent taking unintended real-world actions.
- Data poisoning: adversarial contamination of RAG knowledge base.
- Model extraction: inferring model weights from API outputs.

Monitoring: Input/output classifiers, anomaly detection on output distributions, human review sampling, golden test set regression suite.

System Design

Q77 Design an LLM-powered customer support system for a SaaS company.

ANSWER

Components:

1. Retrieval layer: RAG over product docs, FAQs, historical tickets (pgvector + hybrid search).
2. Intent classification: small model to route: FAQ -> RAG, billing -> tool call, complex -> human escalation.
3. LLM core: GPT-4o or Claude claude-opus-4-6 with system prompt defining persona, policies, escalation rules.
4. Tool integrations: ticket creation (Zendesk API), account lookup, order status.
5. Memory: conversation history stored per session_id; summarise after 10 turns.
6. Safety: output classifier before sending to user; PII redaction from logs.
7. Human handoff: confidence score threshold triggers agent transfer with full context.
8. Analytics: track resolution rate, CSAT, escalation rate, cost/ticket.

Latency target: TTFT <500ms; use streaming for perceived responsiveness.

Q78 Design a document intelligence pipeline that can answer questions over 10,000 internal documents.

ANSWER

Ingestion pipeline:

- Parse: PDFs (PyMuPDF), DOCX, HTML -> clean text + preserve structure/tables.
- Chunk: Recursive text splitter, 512 tokens, 50 token overlap. Tables chunked separately.
- Embed: BGE-M3 or OpenAI text-embedding-3-large -> 1536-dim vectors.
- Store: pgvector (Postgres extension) for combined SQL + vector queries.

Retrieval:

- Hybrid: BM25 (Elasticsearch) + dense (pgvector) -> RRF fusion -> cross-encoder rerank top-20.
- Metadata filters: document type, date, department -> pre-filter before vector search.

Generation:

- Stuff top-5 chunks into context with citations.
- Prompt: 'Answer only from the provided documents. Cite [doc_name, page] for each claim.'
- Stream response with source attribution UI.

Evaluation: RAGAS on golden Q&A; set; weekly regression testing.

Q79 How would you build an LLM-based code review system?**ANSWER**

Pipeline:

1. Trigger: PR webhook -> extract diff (git diff, unified format).
2. Chunking: Split diff by file/function; don't exceed context window.
3. Context augmentation: Retrieve related code context (callers, type definitions) using LSP or tree-sitter AST analysis.
4. Parallel review: For each changed file/function, call LLM with:
 - System prompt: coding standards, security rules, project conventions.
 - User prompt: diff + context + 'List issues with severity: CRITICAL/HIGH/MEDIUM/LOW.'
5. Aggregation: Deduplicate across chunks; filter by severity threshold.
6. Output: Post structured GitHub comments at specific diff lines via GitHub API.

Quality: Fine-tune on accepted/rejected historical PR comments for relevance scoring. Track false positive rate (developers dismissing comments) for feedback loop.

Q80 You need to reduce LLM API costs by 70% without degrading quality significantly. What approaches do you take?**ANSWER**

1. Model routing (highest impact): Classify query complexity. Route simple queries (greetings, factoid, short summaries) to cheap small models (GPT-4o-mini, Claude Haiku). Only send complex tasks to expensive large models. Typical: 60-70% of queries go to small model.
2. Prompt compression: Remove redundant tokens from prompts (LLMLingua achieves 2-5x compression with <5% quality loss). Compress long retrieved contexts.
3. Caching: Semantic cache (GPTCache): 10-30% hit rate on typical workloads; KV-prefix cache for shared system prompts.
4. Batching: Group async requests; use Batch API (50% cost reduction on OpenAI).
5. Output length control: Constrain max_tokens; prompt for concise answers.
6. Self-hosted open models: Deploy LLaMA-3/Mistral on owned GPUs for high-volume tasks. GPU amortises quickly at scale.

Q81 How do you handle PII and data privacy in LLM applications?

ANSWER

Input-side:

- PII detection: Use Presidio, spaCy NER, or a fine-tuned classifier to identify PII (names, SSNs, credit cards, emails, phone numbers) before sending to LLM API.
- Pseudonymisation: Replace detected PII with placeholders (PERSON_1, EMAIL_1) -> send to LLM -> restore in response. Preserves context without exposing raw PII.
- On-premises deployment: Use self-hosted open models (LLaMA, Mistral) so data never leaves your infrastructure.

Output-side:

- Scan outputs for PII before returning to users.
- Prevent memorisation attacks: don't fine-tune on PII-containing data.

Logging:

- Log sanitised prompts/responses; never log raw PII.
- Implement retention policies; GDPR right-to-erasure compliance.
- Role-based access control on LLM logs.

Q82 How would you set up an LLM evaluation and continuous monitoring system in production?

ANSWER

Offline evaluation (pre-deployment):

- Golden test set: 200-500 manually curated (query, reference_answer) pairs covering edge cases and common patterns. Run before every model/prompt change.
- Metrics: LLM-as-judge (GPT-4 grader), RAGAS for RAG, task-specific metrics (F1 for NER).
- Regression detection: Alert if quality drops >5% vs baseline.

Online monitoring (post-deployment):

- Sample 1-5% of production traffic for human review.
- Automated classifiers: safety classifier, refusal rate, response length distribution.
- User signals: thumbs down rate, conversation abandon rate, escalation rate.
- Latency: P50, P95, P99 TTFT and total response time.
- Cost: tokens/request, cost/DAU trend.

Tools: LangSmith, Arize Phoenix, Weights & Biases, Datadog LLM Observability.